

Practical No. 7

Name: Rana Chavan

Section: A4 – B2

Roll no. 26

Aim: Implement Hamiltonian Cycle using Backtracking.

CODE:

```
import java.util.Arrays;

public class HamiltonianCycle {

    private int V;
    private int[][] graph;
    private int[] path;
    private boolean[] visited;
    private String[] vertexNames;

    public HamiltonianCycle(int[][] graph, String[] vertexNames) {
        this.V = graph.length;
        this.graph = graph;
        this.vertexNames = vertexNames;

        this.path = new int[V];
        this.visited = new boolean[V];

        Arrays.fill(path, -1);
    }

    public void findCycle() {
        path[0] = 0;
        visited[0] = true;

        if (!solveCycleRecursive(1)) {
            System.out.println("No Hamiltonian Cycle exists for this graph.");
            System.out.println("-");
        }
    }

    private boolean solveCycleRecursive(int pos) {
        if (pos == V) {
            int lastVertex = path[pos - 1];
            int firstVertex = path[0];

            if (graph[lastVertex][firstVertex] == 1) {
                printSolution();
                return true;
            } else {
                return false;
            }
        }

        for (int v = 1; v < V; v++) {
            if (isSafe(v, pos)) {
                path[pos] = v;
                visited[v] = true;
            }
        }
    }

    private boolean isSafe(int v, int pos) {
        for (int i = 0; i < pos; i++) {
            if (path[i] == v) {
                return false;
            }
        }
        return true;
    }

    private void printSolution() {
        for (int i = 0; i < V; i++) {
            System.out.print(vertexNames[path[i]] + " ");
        }
        System.out.println();
    }
}
```

```

        if (solveCycleRecursive(pos + 1)) {
            return true;
        }

        visited[v] = false;
        path[pos] = -1;
    }
}

return false;
}

private boolean isSafe(int v, int pos) {
    int previousVertex = path[pos - 1];
    if (graph[previousVertex][v] == 0) {
        return false;
    }

    if (visited[v]) {
        return false;
    }

    return true;
}

private void printSolution() {
    System.out.println("Hamiltonian Cycle Found:");
    for (int i = 0; i < V; i++) {
        System.out.print(vertexNames[path[i]] + " -> ");
    }
    System.out.println(vertexNames[path[0]]);
    System.out.println("-");
}

public static void main(String[] args) {

    System.out.println("Testing Graph 1 (A, B, C, D, E):");
    int[][] graph1 = {
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0},
        {1, 1, 0, 1, 0},
        {0, 1, 1, 0, 1},
        {1, 0, 0, 1, 0}
    };
    String[] names1 = {"A", "B", "C", "D", "E"};

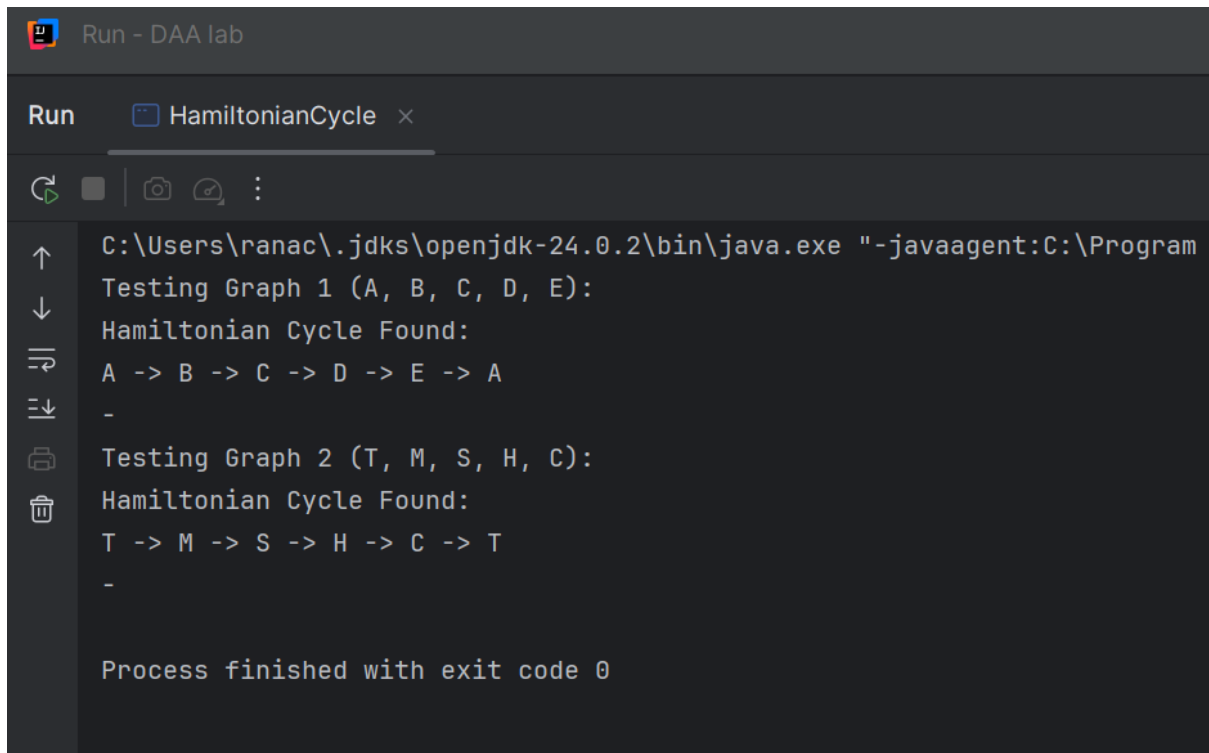
    HamiltonianCycle hc1 = new HamiltonianCycle(graph1, names1);
    hc1.findCycle();

    System.out.println("Testing Graph 2 (T, M, S, H, C):");
    int[][] graph2 = {
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0},
        {1, 1, 0, 1, 1},
        {0, 1, 1, 0, 1},
        {1, 0, 1, 1, 0}
    };
    String[] names2 = {"T", "M", "S", "H", "C"};

    HamiltonianCycle hc2 = new HamiltonianCycle(graph2, names2);
    hc2.findCycle();
}
}

```

OUTPUT:



```
Run - DAA lab
Run  HamiltonianCycle x
C:\Users\ranaac\.jdk\openjdk-24.0.2\bin\java.exe "-javaagent:C:\Program
Testing Graph 1 (A, B, C, D, E):
Hamiltonian Cycle Found:
A -> B -> C -> D -> E -> A
-
Testing Graph 2 (T, M, S, H, C):
Hamiltonian Cycle Found:
T -> M -> S -> H -> C -> T
-
Process finished with exit code 0
```